

Data Structure & Algorithms questions

LeetCode DSA Solution:

[https://github.com/Hugs-4-Bugs/LeetCode-Solution\](https://github.com/Hugs-4-Bugs/LeetCode-Solution)

GeeksForGeeks DSA Solution:

[https://github.com/Hugs-4-Bugs/GeeksForGeeks-POTD-Solution\](https://github.com/Hugs-4-Bugs/GeeksForGeeks-POTD-Solution)

HackerRank DSA Solution:

<https://t.ly/Asnws>

HackerRank 30 days DSA Solution:

[https://github.com/Hugs-4-Bugs/HackerRank--30-Days-of-Code\](https://github.com/Hugs-4-Bugs/HackerRank--30-Days-of-Code)

Portfolio website:

<https://hugs-4-bugs.github.io/myResume/>

Arrays

1. Find the maximum and minimum element in an array.
2. Reverse an array.
3. Move all zeroes to the end of the array.
4. Find the common elements in two arrays.
5. Rotate an array to the right by `k` positions.
6. Find the largest sum contiguous subarray (Kadane's Algorithm).
7. Merge two sorted arrays.
8. Find the intersection of two arrays.
9. Find the missing number in a sequence.
10. Find the duplicate number in an array.
11. Find occurrence of each element
12. Top K element

Strings

1. Reverse a string.
2. Check if a string is a palindrome.
3. Count the number of occurrences of each character.
4. Check if two strings are anagrams.
5. Find the longest substring without repeating characters.
6. Implement string compression.
7. Find the first non-repeating character in a string.
8. Rotate a string by `k` characters.
9. Check if a string contains all unique characters.
10. Find all permutations of a string.
11. Prefix Sum

ArrayList

1. Implement a stack using ArrayList.
2. Implement a queue using ArrayList.
3. Find the maximum and minimum value in an ArrayList.
4. Remove duplicates from an ArrayList.
5. Sort an ArrayList of integers.
6. Find the intersection of two ArrayLists.
7. Reverse an ArrayList.
8. Find the k'th largest element in an ArrayList.
9. Merge two ArrayLists.
10. Find if an ArrayList contains a duplicate element.

Linked Lists (incomplete hai)

1. Reverse a linked list.
2. Detect a cycle in a linked list.
3. Find the middle element of a linked list.
4. Merge two sorted linked lists.
5. Remove duplicates from a linked list.
6. Find the nth node from the end of a linked list.
7. Add two numbers represented by linked lists.
8. Rotate a linked list.
9. Check if a linked list is a palindrome.
10. Flatten a linked list with next and random pointers.

Stacks (Question practice baki hai)

1. Implement a stack using arrays.
2. Implement a stack using linked lists.
3. Check for balanced parentheses in an expression.
4. Evaluate a postfix expression.
5. Find the next greater element for each element in an array.
6. Sort a stack using recursion.
7. Implement a min stack that supports push, pop, top, and retrieving the minimum element.
8. Convert an infix expression to a postfix expression.
9. Implement a stack with a maximum element retrieval.
10. Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Queues ! (Revision + Question practice baki hai)

1. Implement a queue using stacks.
2. Implement a circular queue.
3. Design a data structure that supports the following operations: insert, delete, get_random_element. All operations should be done in constant time.
4. Find the first non-repeating character in a stream of characters.
5. Implement a priority queue.
6. Reverse the first k elements of a queue.
7. Generate binary numbers from 1 to 2^n using a queue.
8. Implement a dequeue (double-ended queue).
9. Implement a queue using two stacks.
10. Implement a sliding window maximum using a queue.

Trees (Question practice: Basic , Medium ! , Adv)

1. Traverse a binary tree in-order, pre-order, and post-order.
2. Find the height of a binary tree.
3. Check if a binary tree is balanced.
4. Find the lowest common ancestor of two nodes.
5. Convert a binary tree to a doubly linked list.
6. Serialize and deserialize a binary tree. 
7. Find all leaf nodes in a binary tree.
8. Check if a binary tree is a subtree of another binary tree.
9. Perform level-order traversal of a binary tree.
10. Construct a binary tree from its inorder and preorder traversal.  (Question practice: Easy , Med , Adv )

List (General) (Leetcode ques. Practice baki hai)

1. Find the intersection of two lists.
2. Find the union of two lists.
3. Remove duplicates from a list.
4. Rotate a list.
5. Find the median of a list.
6. Merge two sorted lists.
7. Find the common elements in multiple lists.
8. Implement a list with custom sorting.
9. Remove every nth element from a list.
10. Reverse a list.

Graphs (Padhna baki hai) [Very imp topic]

1. Implement Depth-First Search (DFS) and Breadth-First Search (BFS). 
2. Find the shortest path in an unweighted graph (BFS). 
3. Find the shortest path in a weighted graph (Dijkstra's Algorithm). 
4. Detect a cycle in a graph.
5. Find connected components in an undirected graph.
6. Topological sorting of a directed graph.
7. Find all paths from a source to a destination in a directed graph.
8. Check if a graph is bipartite.
9. Find the shortest path between all pairs of nodes (Floyd-Warshall Algorithm).
10. Find the Minimum Spanning Tree (Kruskal's and Prim's Algorithm).

Sorting (Revision + Question practice baki hai)

1. Implement Bubble Sort.
2. Implement Merge Sort.
3. Implement Quick Sort.
4. Implement Insertion Sort.
5. Implement Selection Sort.
6. Implement Heap Sort.
7. Implement Counting Sort.
8. Implement Radix Sort.
9. Implement Bucket Sort.
10. Find the kth smallest/largest element in an array.

Searching (Revision + Question practice baki hai)

1. Implement Binary Search.
2. Implement Linear Search.
3. Search in a rotated sorted array.
4. Find the first and last occurrence of an element in a sorted array.
5. Search for a range of elements in a sorted array.
6. Implement Ternary Search.
7. Find the position of a given number in a sorted matrix.
8. Implement exponential search.
9. Find an element in a sorted and rotated array.
10. Find the peak element in an array.

Recursion (Leetcode ques. Practice baki hai)

1. Compute the factorial of a number.
2. Solve the Fibonacci sequence.
3. Implement a recursive binary search.
4. Solve the Tower of Hanoi problem.
5. Generate all permutations of a string.
6. Solve the N-Queens problem.
7. Implement a recursive algorithm to find the power of a number.
8. Print all subsets of a set.
9. Find the sum of all elements in a list using recursion.
10. Generate all possible combinations of a set.
11. Print number 1 to 10 using recursion

Backtracking (Revision + Question practice baki hai) (Leetcode ques. not done)

1. Solve the N-Queens problem. 
2. Solve the Sudoku puzzle. 
3. Generate all permutations of a list. 
4. Find all subsets of a set (power set). 
5. Solve the Rat in a Maze problem. 
6. Find all possible combinations of a given sum. 
7. Generate all possible combinations of a given set.
8. Find all valid combinations of parentheses. 
9. Solve the Word Break problem. 
10. Find all Hamiltonian paths in a graph. 

Dynamic Programming (Padhna baki hai)

1. Compute the Fibonacci sequence using DP.
2. Solve the Knapsack problem.
3. Find the longest common subsequence.
4. Find the longest increasing subsequence.
5. Solve the matrix chain multiplication problem.
6. Find the minimum number of coins for a given amount.
7. Solve the Edit Distance problem.
8. Find the maximum sum of non-adjacent elements.
9. Solve the Rod Cutting problem.
10. Find the minimum path sum in a grid.

Bit Manipulation❌ [Padhna baki hai] (jyada imp nahi hai)

1. Find the single non-duplicate element in an array where every element appears twice.
2. Count the number of set bits in an integer.
3. Find the unique number in an array where every element appears three times except one.
4. Check if a number is a power of two.
5. Find the missing number in an array of 0 to n.
6. Swap two numbers without using a temporary variable.
7. Compute the bitwise AND of a range of numbers.
8. Find the two non-repeating elements in an array where every other element repeats twice.
9. Find the integer that appears an odd number of times in an array.
10. Determine if two integers have opposite signs.

HashMap✅ (LeetCode Question practice: Easy✅, Med✅, Adv❌)

1. Count the frequency of elements in an array.
2. Find the first non-repeating character in a string.
3. Find the intersection of two arrays.
4. Check if two strings are anagrams.
5. Group anagrams from a list of strings.
6. Find the pair with a given sum in an array.
7. Find all pairs in an array that sum up to a given value.
8. Check if a subarray with zero sum exists.
9. Find the longest substring without repeating characters.
10. Find the majority element in an array.

HashSet✅ (LeetCode Question practice: Easy✅, Med✅, Adv❌)

1. Remove duplicates from an array.
2. Find the intersection of two arrays.
3. Find the union of two arrays.
4. Determine if there are any duplicate elements in a list.
5. Find the first missing positive integer.
6. Find the longest consecutive sequence in an unsorted array.
7. Determine if a string contains all unique characters.
8. Check if an array contains duplicate elements within a given range.
9. Find the common elements in multiple arrays.
10. Determine if a set of intervals overlap.

Hashtable ! (LeetCode Question practice: Easy✅, Med❌, Adv❌)

1. Implement a basic hashtable.
2. Find the most frequent element in an array.
3. Determine if two strings are permutations of each other.
4. Check if an array contains a duplicate within a given distance.
5. Find the longest substring with exactly `k` unique characters.
6. Find all pairs of elements that sum up to a specific value.
7. Check if a given array has a subarray with zero sum.
8. Group words that are anagrams together.
9. Implement a cache with limited capacity using hashtable.
10. Find the longest substring with no repeating characters using hashtable.

Binary Search Tree (BST)✅ (Revision & LeetCode Question practice: Easy✅, Med❌, Adv❌) (do only basic operations)

1. Insert a node in a Binary Search Tree.
2. Delete a node in a Binary Search Tree.
3. Search for a value in a Binary Search Tree.
4. Find the minimum and maximum values in a Binary Search Tree.
5. Find the inorder successor and predecessor of a node in a Binary Search Tree.
6. Check if a binary tree is a Binary Search Tree.
7. Find the lowest common ancestor (LCA) of two nodes in a BST.
8. Convert a Binary Search Tree into a sorted doubly linked list.
9. Find the kth smallest element in a Binary Search Tree.
10. Merge two Binary Search Trees.

Trie ! (Revision baki hai) (LeetCode Question practice: Easy❌, Med❌, Adv❌)

1. Implement a Trie (Prefix Tree) from scratch.
2. Insert a word into a Trie.
3. Search for a word in a Trie.
4. Delete a word from a Trie.
5. Check if a word starts with a given prefix.
6. Count the number of words in a Trie with a given prefix.
7. Find all words in a Trie that start with a given prefix.
8. Implement autocomplete suggestions using a Trie.
9. Find the longest common prefix of a list of words using a Trie.
10. Implement a spell checker using a Trie.

Miscellaneous

1. Implement LRU Cache. 
2. Find the maximum product of two integers in an array. 
3. Compute the greatest common divisor (GCD) of two numbers. 
4. Find the nth Fibonacci number. 
5. Check if a number is prime. 
6. Generate all possible combinations of a given set. 
7. Implement a basic calculator. 
8. Determine the number of islands in a 2D grid. 
9. Find the longest increasing subsequence in a list. 
10. Calculate the power set of a given set. 

ALGORITHMS:

Greedy  (Leetcode ques. Practice baki hai)
Sorting 
Searching 
Two Pointer  (Leetcode ques. Practice baki hai)
Dynamic Programming  (padhna baki hai)
Brute-force (Leetcode ques. Practice baki hai) 
DjKastr's (also known as Dijkstra's) 
Recursion
Taken & not taken 
Memorization 
Tabulation 
Divide and Conquer 
String Manipulation 
Kadane's algorithm 
Sliding windows 
Stock Buy & Sell  (Leetcode ques. Practice baki hai)
Topological Sorting  (Leetcode ques. Practice baki hai)

Sorting Algorithms

- Bubble sort 
- Selection sort 
- Insertion sort 
- Merge sort 
- Quick sort 
- Heap sort 
- Radix sort 
- Timsort 
- Counting sort 
- Bucket sort 
- Shell sort 

Searching Algorithms

- Linear search
- Binary search
- Depth-first search (DFS)
- Breadth-first search (BFS)

Graph Algorithms

- Dijkstra's algorithm (also known as DjKastra's)
- Bellman-Ford algorithm
- Floyd-Warshall algorithm
- Topological sort
- Kruskal's algorithm
- Prim's algorithm
- Breadth-first search (BFS) 
- Depth-first search (DFS) 

Dynamic Programming Algorithms (Padhna baki hai)

- Fibonacci series 
- Longest common subsequence
- Shortest path problem
- Knapsack problem
- Matrix chain multiplication
- Longest increasing subsequence

Greedy Algorithms

- Huffman coding
- Activity selection problem
- Fractional knapsack problem
- Greedy coloring

Two Pointer Algorithms

- Two sum problem
- Three sum problem
- Container with most water
- Trapping rain water 

Brute-force Algorithms (LeetCode Question practice baki hai)

- Brute-force search
- Brute-force string matching

Recursion Algorithms (LeetCode Question practice baki hai)

- Factorial
- Fibonacci series
- Tower of Hanoi  (revision Karna hai) [Jyada imp phi nahi hai]
- Merge sort
- Quick sort

Taken & Not Taken Algorithms (LeetCode Question practice baki hai)

- 0/1 Knapsack problem
- Subset sum problem

Memorization Algorithms

- Memoized Fibonacci series
- Memoized longest common subsequence

Tabulation Algorithms

- Tabulated Fibonacci series
- Tabulated longest common subsequence

Divide and Conquer Algorithms (LeetCode Question practice baki hai)

- Merge sort
- Quick sort
- Binary search
- Closest pair problem 

String Manipulation Algorithms

- Rabin-Karp algorithm 
- Knuth-Morris-Pratt algorithm 
- Longest palindromic substring 
- Longest common prefix 

Kadane Algorithm

- Maximum subarray problem

Sliding Windows Algorithms (LeetCode Question practice baki hai)

- Maximum sum subarray problem
- Minimum window substring problem 

Stock Buy & Sell Algorithms (LeetCode Question practice baki hai)

- Best time to buy and sell stock
- Best time to buy and sell stock II

Topological Sorting Algorithms

- Kahn's algorithm 
- Depth-first search (DFS) 

Backtracking Algorithms

- N-Queens problem 
- Sudoku 
- Knapsack problem 

Cryptography Algorithms [NAHI PADHNA HAI]

- RSA
- AES
- SHA-256

Machine Learning Algorithms

- Linear regression 
- Logistic regression 
- Decision tree 
- Random forest 
- K-means clustering 
- K-nearest neighbors (KNN) 
- Re-current nearest neighbors(RNN) 
- LSTM 
- Reinforcement learning (RL) 
- A* 
- AO* 
- DFS 
- BFS 
- Learning task & Q Learning 
- Expectation-Maximization (EM) 